

Explainability of Quantum Layers in Physics-Informed Neural Networks:

A Study of Quantum-Assisted PINNs for Nonlinear and Linear PDEs

Mehrdad Ghanbari Mobarakeh, Fataneh Bakherad

August 2026

Abstract

Physics-Informed Neural Networks (PINNs) provide a powerful framework for solving partial differential equations (PDEs) by embedding physical laws into the training objective. However, classical PINNs suffer from optimization difficulties, spectral bias, and limited representation capacity for complex solution landscapes.

This work investigates how the introduction of a variational quantum circuit changes the learning behavior of PINNs. Instead of attempting to demonstrate that Quantum-Assisted Physics-Informed Neural Networks (QAPINNs) universally outperform classical PINNs, we analyze when and why a quantum layer modifies the optimization dynamics, parameter efficiency, and PDE approximation capability.

A hybrid quantum-classical architecture is developed using PennyLane and PyTorch. The quantum component employs data re-uploading, trainable single-qubit rotations, ring entanglement, and expectation-value measurements. The proposed framework is evaluated on two benchmark PDEs: the nonlinear Allen–Cahn equation and the linear Heat equation.

The experimental results demonstrate that quantum layers can significantly reduce the number of trainable parameters while maintaining competitive accuracy. However, increased circuit depth does not always improve performance and may introduce optimization difficulties. The results indicate that the effectiveness of a quantum layer depends on the relationship between PDE complexity, circuit expressivity, qubit number, and trainability.

1 Introduction

Physics-Informed Neural Networks (PINNs) have emerged as an important approach for solving differential equations by combining neural network approximation with physical constraints. Instead of learning only from available data, PINNs minimize a composite loss function containing the governing PDE residual, initial conditions, and boundary conditions. This physics-based training strategy enables the solution of forward and inverse problems even when measurement data are limited.

Despite their success, classical PINNs face several challenges. The nonlinear optimization landscape can contain poorly conditioned regions, high-frequency solution components are difficult to represent due to neural spectral bias, and increasing model capacity often requires significantly more trainable parameters.

Recent developments in quantum machine learning suggest that variational quantum circuits (VQCs) may provide alternative nonlinear feature mappings. A quantum layer can transform classical inputs into a higher-dimensional quantum feature space where trainable rotations and entanglement generate complex representations.

However, the key scientific question is not whether a quantum model simply achieves a lower error. The more fundamental question is:

When, why, and how does the introduction of a quantum layer modify the learning dynamics of a physics-informed neural network?

This work addresses this question through systematic experiments involving different numbers of qubits and circuit depths. The analysis focuses on:

- the effect of quantum layers on PDE approximation accuracy,
- the relationship between circuit depth and trainability,
- parameter reduction compared with classical PINNs,
- computational cost introduced by quantum simulation,
- guidelines for designing problem-specific quantum circuits.

The main contributions of this work are:

1. Development of a hybrid QAPINN architecture for solving nonlinear and linear PDEs.
2. Systematic comparison of classical PINNs and QAPINNs using Allen–Cahn and Heat equation benchmarks.
3. Analysis of how qubit number and circuit depth influence learning behavior.
4. Proposal of a methodology for selecting quantum circuit structures based on PDE characteristics.

The remainder of this paper is organized as follows. Section 2 introduces the mathematical formulation of PINNs and QAPINNs. Section 3 describes the quantum circuit design and experimental methodology. Section 4 presents Allen–Cahn results, while Section 5 evaluates the Heat equation benchmark. Section 6 discusses explainability and quantum learning dynamics. Finally, Section 7 summarizes conclusions and future directions.

2 Mathematical Background and QAPINN Formulation

2.1 Physics-Informed Neural Networks

Consider a general partial differential equation:

$$\mathcal{F}[u(x, t)] = 0, \quad (1)$$

where $u(x, t)$ represents the unknown solution and $\mathcal{F}$ denotes the differential operator associated with the physical system.

A PINN approximates the solution using a neural network:

$$u_\theta(x, t) \approx u(x, t), \quad (2)$$

where θ represents the trainable network parameters. Automatic differentiation is used to compute derivatives required by the governing PDE.

The training objective is constructed by minimizing the weighted combination of three physics-based losses:

$$\mathcal{L} = \lambda_f \mathcal{L}_f + \lambda_i \mathcal{L}_i + \lambda_b \mathcal{L}_b, \quad (3)$$

where $\mathcal{L}_f$ is the PDE residual loss, $\mathcal{L}_i$ enforces the initial condition, and $\mathcal{L}_b$ satisfies boundary constraints.

The PDE residual term is defined as:

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{F}[u_\theta(x_i, t_i)]|^2. \quad (4)$$

2.2 Quantum-Assisted Physics-Informed Neural Network

A QAPINN replaces part of the classical feature transformation with a parameterized quantum circuit. The proposed architecture consists of three components:

$$(x, t) \rightarrow \text{Encoder} \rightarrow \text{Quantum Layer} \rightarrow \text{Decoder} \rightarrow u_\theta(x, t). \quad (5)$$

2.2.1 Classical Encoder

The encoder maps the two-dimensional space-time coordinates into a quantum feature vector:

$$z = E_\phi(x, t), \quad (6)$$

where E_ϕ is a classical neural network.

The implemented encoder is:

$$(2) \rightarrow \text{Linear}(32) \rightarrow \text{Tanh} \rightarrow \text{Linear}(N_q), \quad (7)$$

where N_q is the number of qubits.

The encoder learns a continuous representation suitable for quantum processing.

2.2.2 Variational Quantum Layer

The quantum layer performs a trainable nonlinear transformation:

$$q = Q_\theta(z), \quad (8)$$

where Q_θ represents the variational quantum circuit.

The circuit uses data re-uploading, where the encoded classical information is reintroduced at every quantum layer:

$$RY(\pi z_q). \quad (9)$$

This repeated encoding increases the effective feature frequency and improves the ability of the model to represent complex PDE solutions.

Each variational layer applies trainable single-qubit rotations:

$$U(\theta) = RY(\theta_y)RZ(\theta_z). \quad (10)$$

The complete circuit transformation is therefore:

$$|\psi\rangle = \prod_{l=1}^L U_l(\theta_l) E_l(z) |\psi_0\rangle. \quad (11)$$

2.2.3 Entanglement Structure

To introduce correlations between quantum variables, a periodic ring entanglement pattern is employed.

For N_q qubits:

$$CNOT(q, q + 1), \quad (12)$$

with an additional connection:

$$CNOT(N_q - 1, 0). \quad (13)$$

This topology allows information exchange between all qubits while maintaining a relatively shallow circuit depth.

2.2.4 Measurement Operator

The quantum state is converted into classical features using expectation-value measurements:

$$q_i = \langle \psi | Z_i | \psi \rangle. \quad (14)$$

The output satisfies:

$$q_i \in [-1, 1], \quad (15)$$

and produces an interpretable bounded feature vector:

$$q = (q_1, q_2, \dots, q_{N_q}). \quad (16)$$

2.2.5 Classical Decoder

The decoder reconstructs the PDE solution from the quantum features:

$$u_\theta(x, t) = D_\omega(q). \quad (17)$$

The implemented decoder consists of:

$$(N_q) \rightarrow \text{Linear}(32) \rightarrow \text{Tanh} \rightarrow \text{Linear}(1). \quad (18)$$

2.3 Parameter Scaling

One important motivation for introducing a quantum layer is parameter efficiency.

The number of quantum parameters scales as:

$$P_Q = 2LN_q, \quad (19)$$

where L is the circuit depth and N_q is the number of qubits.

Unlike classical networks, increasing representation capacity can therefore be achieved by modifying the quantum feature map rather than continuously expanding fully connected layers.

2.4 Benchmark PDEs

Two PDE benchmarks are considered in this work.

2.4.1 Allen–Cahn Equation

The nonlinear Allen–Cahn equation is defined as:

$$u_t - \epsilon^2 u_{xx} + u^3 - u = 0, \quad (20)$$

where ϵ controls interface sharpness.

This equation represents a challenging nonlinear system because of its steep transition regions and high-frequency solution components.

2.4.2 Heat Equation

The one-dimensional Heat equation is:

$$u_t = \alpha u_{xx}. \quad (21)$$

Unlike Allen–Cahn, this PDE has linear diffusion dynamics and provides a simpler benchmark for analyzing the effect of the quantum layer.

3 Experimental Setup and Methodology

3.1 Implementation Framework

All experiments were performed using a hybrid quantum-classical simulation framework based on PyTorch and PennyLane. The quantum circuits were simulated using the state-vector simulator backend. The objective was not to evaluate quantum hardware performance, but to analyze how the presence of a quantum feature transformation affects PINN learning dynamics.

All models were trained using deterministic random seeds to ensure reproducibility. Double precision floating point arithmetic was used for the physics-informed calculations because PDE residual evaluation requires accurate second-order derivatives.

3.2 Classical PINN Architecture

The classical baseline model is a fully connected neural network:

$$(2) \rightarrow \text{Linear}(32) \rightarrow \text{Tanh} \rightarrow \text{Linear}(32) \rightarrow \text{Tanh} \rightarrow \text{Linear}(1). \quad (22)$$

The input variables are the spatial and temporal coordinates (x, t) and the output is the predicted PDE solution $u(x, t)$.

The activation function was selected as hyperbolic tangent because PINN training requires stable higher-order derivatives:

$$\frac{\partial^2 u}{\partial x^2}. \quad (23)$$

Unlike ReLU activations, the smoothness of the tanh function allows stable automatic differentiation.

3.3 QAPINN Architecture

The QAPINN model contains a classical encoder, a variational quantum circuit, and a classical decoder.

The complete transformation is:

$$(x, t) \xrightarrow{E_\phi} z \xrightarrow{Q_\theta} q \xrightarrow{D_\omega} u(x, t). \quad (24)$$

The encoder compresses the physical coordinates into a quantum-compatible feature representation:

$$E_\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^{N_q}. \quad (25)$$

The quantum circuit maps this representation into expectation values:

$$Q_\theta : \mathbb{R}^{N_q} \rightarrow [-1, 1]^{N_q}. \quad (26)$$

The decoder converts the quantum features back into the PDE solution space.

3.4 Training Configuration

All models were optimized using the Adam optimizer:

$$\eta = 10^{-3}. \quad (27)$$

A learning-rate scheduler was applied to improve convergence by reducing the learning rate when the loss stagnated.

The main training parameters are summarized in Table 1.

Table 1: Training configuration used for all experiments.

Parameter	Value
Optimizer	Adam
Initial learning rate	10^{-3}
Activation function	tanh
Training epochs	3000–8000
Quantum simulator	PennyLane default.qubit
Automatic differentiation	PyTorch interface
Sampling method	Latin Hypercube Sampling

3.5 Physics-Based Sampling

The PINN loss was evaluated using three groups of points:

- Collocation points sampled inside the PDE domain to minimize the governing equation residual.
- Initial-condition points enforcing the known initial solution.
- Boundary-condition points enforcing spatial constraints.

The complete loss function is:

$$\mathcal{L} = \mathcal{L}_{PDE} + \mathcal{L}_{IC} + \mathcal{L}_{BC}. \quad (28)$$

3.6 Evaluation Metrics

The following metrics were used to compare classical PINNs and QAPINNs.

3.6.1 Relative L_2 Error

The solution accuracy was evaluated using:

$$L_2 = \frac{\|u_{pred} - u_{true}\|}{\|u_{true}\|}. \quad (29)$$

3.6.2 PDE Residual Error

The physics consistency was measured using the residual:

$$RMSE_{PDE} = \sqrt{\frac{1}{N} \sum_i |\mathcal{F}(u_i)|^2}. \quad (30)$$

3.6.3 Parameter Efficiency

The total number of trainable parameters was recorded:

$$P_{total} = P_{classical} + P_{quantum}. \quad (31)$$

This metric is important because one possible advantage of quantum layers is increasing representation capacity without increasing classical network size.

3.6.4 Computational Cost

Training time and memory requirements were measured to quantify the additional computational overhead introduced by quantum simulation.

3.7 Quantum Circuit Design Methodology

A major objective of this work is to understand how a problem-specific quantum circuit should be designed.

The proposed design principles are:

- 1. Continuous PDE variables require continuous encoding.**

The space-time coordinates are injected through rotation angles rather than discrete basis encoding.

- 2. Nonlinear PDEs benefit from repeated encoding.**

Data re-uploading increases the effective Fourier representation of the quantum feature map and allows the circuit to represent sharper solution structures.

- 3. Entanglement should match physical correlations.**

The ring entanglement structure allows information exchange between all variables while avoiding excessive circuit depth.

- 4. Circuit depth should be optimized rather than maximized.**

Increasing layers increases expressivity but may also increase optimization difficulty and gradient instability.

Therefore, the optimal quantum architecture is not necessarily the deepest or largest circuit, but the circuit whose expressivity matches the complexity of the underlying PDE.

4 Allen–Cahn Equation: Experimental Results and Analysis

The Allen–Cahn equation was selected as the primary nonlinear benchmark because it contains nonlinear reaction dynamics and sharp transition regions. These characteristics provide a challenging test for evaluating whether a quantum feature map can modify PINN learning behavior.

The classical PINN baseline was compared against QAPINNs with different numbers of qubits and circuit depths. The purpose of these experiments was not to demonstrate universal quantum superiority, but to determine the conditions under which the quantum layer improves or degrades learning performance.

4.1 Baseline Classical PINN

The classical PINN achieved the following performance:

Table 2: Classical PINN performance for the Allen–Cahn equation.

Model	Parameters	Relative L_2 Error	PDE RMSE
PINN	1185	0.048924	0.013856

The classical model provides a strong baseline with relatively low computational cost. However, it requires a significantly larger number of trainable parameters compared with the compact quantum-assisted models.

4.2 Effect of Qubit Number and Circuit Depth

Table 3 summarizes all Allen–Cahn experiments.

Table 3: Allen–Cahn comparison between PINN and QAPINN architectures.

Model	Qubits	Layers	Parameters	Relative L_2	PDE RMSE	Time(s)
PINN	0	-	1185	0.0489	0.0139	57.8
QAPINN	2	1	295	0.1093	0.0344	276.9
QAPINN	2	2	299	0.0450	0.0299	460.8
QAPINN	2	3	303	0.0475	0.0414	644.5
QAPINN	6	1	563	0.00031	0.00038	5925.3
QAPINN	6	2	575	0.00150	0.00191	11014.6

4.3 Influence of Qubit Number

The number of qubits strongly affects the representational capacity of the QAPINN.

For the two-qubit configuration, the one-layer circuit produced inferior performance compared with the classical PINN. The limited quantum feature space restricts the ability of the model to represent the nonlinear Allen–Cahn solution.

Increasing the circuit depth from one to two layers significantly improved the solution accuracy:

$$L_2 : 0.1093 \rightarrow 0.0450. \quad (32)$$

This demonstrates that increasing the number of variational transformations can partially compensate for the limited number of quantum degrees of freedom.

However, adding a third layer did not provide additional improvement:

$$0.0450 \rightarrow 0.0475. \quad (33)$$

This indicates that increasing circuit depth does not automatically improve learning. Additional layers increase the complexity of the optimization landscape and may introduce trainability

difficulties.

4.4 High-Expressivity Quantum Configuration

The six-qubit one-layer QAPINN achieved the best performance:

$$L_2 = 3.1 \times 10^{-4}, \quad (34)$$

which is significantly lower than the classical PINN error.

This result is important because the improvement was obtained with fewer total parameters:

$$P_{QAPINN} = 563$$

compared with:

$$P_{PINN} = 1185.$$

Therefore, the quantum layer increased the effective representation capacity without requiring a larger classical neural network.

The result suggests that increasing the dimensionality of the quantum feature space can be more beneficial than simply increasing circuit depth.

4.5 Circuit Depth versus Performance

The experiments reveal a non-monotonic relationship between circuit depth and accuracy.

For six qubits:

Configuration	L_2
$6q - 1layer$	3.1×10^{-4}
$6q - 2layer$	1.5×10^{-3}

Although the two-layer circuit contains twice the quantum parameters:

$$12 \rightarrow 24,$$

the performance decreases.

This observation demonstrates that quantum expressivity and trainability must be balanced. Excessive circuit depth can make optimization more difficult due to highly oscillatory parameter landscapes and reduced gradient quality.

4.6 Parameter Efficiency Analysis

A major advantage observed in the QAPINN models is parameter compression.

The best Allen–Cahn configuration achieved:

$$\frac{563}{1185} = 0.475$$

of the classical parameter count.

Therefore, approximately 52.5% fewer trainable parameters were required while obtaining improved accuracy.

This indicates that the quantum layer acts as a compact nonlinear feature transformer rather than simply increasing the number of model parameters.

4.7 Interpretation of Allen–Cahn Learning Dynamics

The Allen–Cahn experiments lead to several conclusions:

1. Small quantum circuits may underperform classical PINNs because their feature space is insufficient for nonlinear PDE structures.
2. Increasing qubit number improves expressivity more effectively than increasing circuit depth.
3. The optimal quantum circuit is problem dependent. The best configuration was not the deepest circuit, but the circuit providing sufficient expressivity with stable optimization.
4. Quantum layers introduce a trade-off:

$$\text{higher representation efficiency} \leftrightarrow \text{higher simulation cost.}$$

5 Heat Equation: Experimental Results and Analysis

The Heat equation was selected as a second benchmark to investigate whether the behavior observed in the nonlinear Allen–Cahn problem also appears for a linear diffusion equation.

Unlike Allen–Cahn, the Heat equation does not contain nonlinear reaction terms. Therefore, it provides a useful test for understanding how quantum layers affect PDEs with smoother solution landscapes.

The governing equation is:

$$u_t = \alpha u_{xx}. \quad (35)$$

The same QAPINN architecture and training methodology were applied without changing the circuit design. The only modification was the PDE residual used in the physics-informed loss.

5.1 Classical PINN Baseline

The classical PINN achieved:

Table 4: Classical PINN performance for the Heat equation.

Model	Parameters	Relative L_2	PDE RMSE	Extrapolation L_2	Time(s)
PINN	3297	0.00214	0.00546	0.0212	97.2

The classical PINN provides a strong reference solution due to the relatively smooth structure of the Heat equation.

5.2 Quantum-Assisted Heat Equation Results

The QAPINN configurations are summarized in Table 5.

Table 5: Heat equation QAPINN performance for different circuit configurations.

Model	Qubits	Layers	Parameters	Relative L_2	PDE RMSE	Time(s)
QAPINN	2	1	10	0.0911	0.2344	624.8
QAPINN	4	1	18	0.1623	0.2539	2045.6
QAPINN	2	3	26	0.0100	0.0188	1621.9
QAPINN	4	3	50	0.00451	0.0225	5349.0

5.3 Effect of Circuit Depth

The Heat equation results show a different trend compared with Allen–Cahn.

For the two-qubit circuit:

$$L_2 : 0.0911 \rightarrow 0.0100$$

when the circuit depth increased from one layer to three layers.

Similarly, the four-qubit model improved substantially:

$$0.1623 \rightarrow 0.00451.$$

This indicates that for smooth diffusion problems, increasing the number of variational transformations can improve the representation of the solution.

The reason is that the Heat equation has a simpler frequency structure than Allen–Cahn. Additional circuit layers can therefore increase approximation capacity without creating severe optimization difficulties.

5.4 Comparison Between Allen–Cahn and Heat Equation

The two PDE benchmarks demonstrate that the optimal quantum architecture depends on the physical characteristics of the problem.

Table 6: Qualitative comparison of quantum layer behavior for different PDEs.

Characteristic	Allen–Cahn	Heat equation
PDE type	Nonlinear reaction-diffusion	Linear diffusion
Solution structure	Sharp interfaces	Smooth evolution
Most effective strategy	Increase qubits	Increase depth
Main challenge	Representation of nonlinear features	Approximation accuracy

For Allen–Cahn, increasing qubit number produced the largest improvement. The six-qubit one-layer circuit achieved the best accuracy because the expanded quantum feature space was able to capture nonlinear solution structures.

For the Heat equation, deeper circuits were more effective. The smoother PDE solution benefited from additional variational transformations rather than additional quantum dimensions.

These results support the hypothesis that quantum circuit design should be problem dependent rather than universally fixed.

5.5 Generalization and Extrapolation Analysis

The models were also evaluated beyond the training time interval to examine generalization.

For the Heat equation, the extrapolation test extended the prediction domain from:

$$t \in [0, 1]$$

to:

$$t \in [1, 1.5].$$

The classical PINN achieved:

$$L_2^{extra} = 2.12 \times 10^{-2}.$$

The QAPINN models showed larger extrapolation errors, especially for shallow circuits. For example:

$$L_2^{extra} = 1.33 \times 10^{-1}$$

for the two-qubit one-layer circuit.

Although QAPINNs can provide compact and expressive feature maps, the results indicate that improved interpolation accuracy does not automatically guarantee better extrapolation capability.

This observation is important because quantum layers modify the representation space, but the learned physical evolution outside the training domain still depends strongly on the optimization process and circuit configuration.

5.6 Computational Cost Trade-Off

A consistent observation across all experiments is the significant increase in training time caused by quantum simulation.

For example, the best Allen–Cahn configuration:

$$6q - 1layer$$

required approximately:

$$5925 \text{ s},$$

while the classical PINN required approximately:

$$58 \text{ s}.$$

This overhead originates from the repeated state-vector simulation and quantum gradient evaluation during backpropagation.

Therefore, the current advantage of QAPINNs is not computational speed, but parameter efficiency and alternative representation capability.

6 Explainability Analysis of the Quantum Layer

The central objective of this work is not to demonstrate that a quantum-assisted PINN universally outperforms a classical PINN, but rather to understand how the introduction of a variational quantum circuit modifies the learning dynamics. Therefore, several explainability metrics were investigated, including parameter efficiency, gradient behavior, quantum feature representation, circuit depth effects, and generalization capability.

6.1 Parameter Efficiency

One of the primary motivations for introducing quantum layers into neural networks is the possibility of representing complex nonlinear functions using compact parameterized circuits. In the investigated QAPINN architectures, the quantum layer replaces a significant portion of classical nonlinear processing.

For the Allen–Cahn equation, the classical PINN contains approximately 1185 trainable parameters. In comparison, the hybrid models require considerably fewer parameters. For example, the six-qubit one-layer QAPINN contains only 563 parameters while achieving the lowest observed final loss among the tested architectures.

This indicates that the quantum layer can provide a compact nonlinear feature mapping. However, parameter reduction does not directly translate into improved accuracy. The quantum circuit introduces additional optimization difficulty and simulation overhead, meaning that a smaller parameter space may require a more careful circuit design.

The results suggest that the role of the quantum layer is not simply parameter compression, but rather transformation of the feature representation learned by the network.

6.2 Effect of Circuit Depth

Circuit depth strongly influences the learning dynamics of the QAPINN. Increasing the number of variational layers increases the expressivity of the quantum mapping because more nonlinear transformations and entangling operations are introduced.

However, the experiments demonstrate that increasing depth does not always improve performance.

For the Allen–Cahn equation:

- The two-qubit QAPINN improved when increasing depth from one to two layers.
- Further increasing the depth to three layers degraded performance.
- The six-qubit one-layer model achieved the best accuracy, while adding an additional layer reduced performance.

This behavior indicates that deeper circuits introduce a trade-off between expressivity and trainability. Additional layers increase the possible function space, but they also increase the complexity of the optimization landscape.

A deeper circuit may therefore introduce redundant degrees of freedom that are not required for the PDE solution.

6.3 Gradient Dynamics and Trainability

A major concern in variational quantum algorithms is the barren plateau phenomenon, where gradients become exponentially small as circuit depth and system size increase.

The training procedure monitored gradient magnitudes of the trainable parameters during optimization. The objective was to determine whether the investigated circuits suffered from vanishing gradients.

The tested circuits used:

- shallow hardware-efficient structures,
- local ring entanglement,
- data re-uploading,
- limited numbers of qubits.

These design choices reduce the probability of entering highly concentrated regions of Hilbert space associated with barren plateaus.

The observed training behavior suggests that the employed circuits remain trainable for small-scale PDE problems. However, the computational cost grows rapidly with the number of qubits and layers due to quantum state-vector simulation.

6.4 Quantum Feature Representation

The quantum layer transforms encoded space-time coordinates into expectation values:

$$q_i = \langle \psi(x, t, \theta) | Z_i | \psi(x, t, \theta) \rangle, \quad (36)$$

where Z_i represents the Pauli-Z measurement operator of qubit i .

The resulting quantum feature vector satisfies:

$$q_i \in [-1, 1]. \quad (37)$$

This bounded representation provides a nonlinear feature space before the final classical decoder. Unlike a conventional neural layer, the transformation is generated through rotations and entanglement operations in a Hilbert space.

For PDE solutions, this feature transformation is particularly relevant because the solution function is not represented directly. Instead, the quantum circuit learns a coordinate transformation that modifies how the classical decoder approximates the physical solution.

6.5 Generalization and Extrapolation

To investigate whether the quantum layer improves physical generalization, models were evaluated outside the training interval.

For the Heat equation, models were trained in:

$$0 \leq t \leq 1$$

and evaluated up to:

$$t = 1.5.$$

The classical PINN achieved an extrapolation error of:

$$L_2^{extra} = 2.12 \times 10^{-2}.$$

The tested QAPINN models showed mixed behavior. The two-qubit one-layer architecture produced a significantly larger extrapolation error:

$$L_2^{extra} = 1.33 \times 10^{-1},$$

while increasing circuit expressivity improved interpolation accuracy but did not necessarily improve extrapolation.

For example, the four-qubit three-layer Heat equation QAPINN achieved:

$$L_2 = 4.51 \times 10^{-3},$$

inside the training domain, but its extrapolation error increased to:

$$L_2^{extra} = 2.22 \times 10^{-1}.$$

This demonstrates that quantum feature mappings may increase approximation capacity but do not automatically guarantee improved physical generalization.

6.6 What is Gained and What is Lost

The introduction of a quantum layer provides several potential advantages:

- Reduced number of trainable parameters.
- Nonlinear feature transformations through quantum evolution.
- Additional design flexibility through encoding and measurement choices.
- Potential suitability for problems requiring complex feature spaces.

However, several disadvantages were also observed:

- Significantly increased training time due to quantum simulation.
- Strong dependence on circuit architecture.
- Deeper circuits may reduce trainability.
- Increased difficulty in selecting optimal qubit numbers and layers.
- No guaranteed improvement in extrapolation behavior.

Therefore, the quantum layer should be considered as an alternative feature mapping mechanism rather than a direct replacement for classical neural components.

7 Problem-Specific QAPINN Design Methodology

A major objective of this study is to establish a systematic methodology for designing quantum-assisted physics-informed neural networks rather than using a fixed quantum architecture for all differential equations. The optimal quantum layer configuration depends on the mathematical properties of the target PDE, including nonlinearity, smoothness, frequency content, and boundary behavior.

The proposed methodology consists of five main design decisions:

1. Selection of the classical encoder-decoder structure.
2. Choice of data encoding strategy.
3. Selection of quantum circuit depth and qubit number.
4. Selection of entanglement topology.
5. Selection of measurement operators.

7.1 Relation Between PDE Complexity and Quantum Expressivity

A physics-informed neural network approximates the unknown solution:

$$u(x, t) \approx u_\theta(x, t), \quad (38)$$

where the network parameters are optimized by minimizing the physics-informed objective:

$$\mathcal{L} = \lambda_f \mathcal{L}_{PDE} + \lambda_i \mathcal{L}_{IC} + \lambda_b \mathcal{L}_{BC}. \quad (39)$$

The quantum-assisted model introduces an intermediate nonlinear transformation:

$$u_\theta(x, t) = D_\phi(Q_\omega(E_\eta(x, t))), \quad (40)$$

where:

- E_η is the classical encoder,
- Q_ω is the variational quantum circuit,
- D_ϕ is the classical decoder.

The quantum layer therefore changes the optimization problem by replacing part of the classical nonlinear approximation with a quantum feature map.

The required expressivity depends on the PDE characteristics.

For smooth PDE solutions, such as the linear Heat equation, shallow circuits may be sufficient. For nonlinear equations with sharp transitions, such as the Allen–Cahn equation, additional nonlinear feature transformations may be beneficial.

7.2 Encoding Strategy Selection

The choice of encoding method determines how physical coordinates are mapped into the quantum state.

This work uses data re-uploading encoding:

$$R_Y(\pi z_q), \quad (41)$$

where z_q represents the encoded feature corresponding to qubit q .

Unlike one-shot encoding, data re-uploading injects the classical information at each circuit layer:

$$|\psi_L\rangle = U_L(x, t, \theta_L) \dots U_2(x, t, \theta_2) U_1(x, t, \theta_1) |0\rangle. \quad (42)$$

This structure increases the frequency components that can be represented by the quantum model.

From a Fourier interpretation, repeated encoding creates higher-order nonlinear basis functions:

$$f(x, t) = \sum_k c_k e^{ik(x, t)}. \quad (43)$$

Therefore:

- Smooth PDEs require fewer encoding repetitions.
- High-frequency nonlinear PDE solutions benefit from additional re-uploading layers.

This explains why the Allen–Cahn equation benefits from carefully selected quantum depth, while excessive depth can introduce unnecessary optimization complexity.

7.3 Qubit Number Selection

The number of qubits controls the dimensionality of the quantum feature space:

$$q \in [-1, 1]^N, \quad (44)$$

where N is the number of qubits.

Increasing the number of qubits provides:

- larger latent feature representations,
- increased entanglement capacity,
- higher possible function complexity.

However, increasing qubit number also increases:

- simulation cost,

- optimization difficulty,
- probability of poor gradient behavior.

The Allen–Cahn experiments demonstrate this trade-off. Increasing from two to six qubits reduced the best observed error:

$$L_2(2Q, 2L) = 4.50 \times 10^{-2},$$

while:

$$L_2(6Q, 1L) = 3.84 \times 10^{-4}.$$

However, increasing both qubit number and depth was not always beneficial:

$$6Q, 2L$$

performed worse than:

$$6Q, 1L.$$

Therefore, the recommended strategy is:

Increase qubits before increasing circuit depth.

For PDEs with low-dimensional smooth solutions, two to four qubits may provide sufficient representation capacity. More complex nonlinear PDEs may require larger quantum feature spaces.

7.4 Entanglement Architecture

The entanglement pattern determines how information is distributed between qubits.

This work uses periodic ring entanglement:

$$CNOT(q, q + 1), \tag{45}$$

with an additional connection:

$$CNOT(N - 1, 0). \tag{46}$$

The ring topology was selected because PDE solutions naturally contain correlations between spatial and temporal variables.

Compared with fully connected entanglement, ring structures provide:

- lower circuit depth,
- reduced simulation cost,
- sufficient correlation propagation for low-dimensional PDEs.

For larger-scale problems, more expressive entanglement structures could be investigated, including:

- hardware-efficient layouts,
- alternating linear connections,
- problem-inspired coupling graphs.

7.5 Measurement Operator Selection

The final quantum state is converted into classical features through measurement. This work uses Pauli-Z expectation values:

$$q_i = \langle \psi | Z_i | \psi \rangle. \quad (47)$$

The resulting feature vector:

$$\mathbf{q} = (q_1, q_2, \dots, q_N) \quad (48)$$

is passed into the classical decoder.

Expectation-value measurements were selected because they provide:

- continuous outputs,
- differentiability,
- compatibility with gradient-based optimization.

Alternative measurement strategies, such as probability amplitudes or complete state tomography, may provide richer information but introduce additional computational complexity.

7.6 Recommended Design Procedure

Based on the experimental observations, the following workflow is proposed:

1. Analyze PDE properties:
 - linear versus nonlinear behavior,
 - smoothness,
 - presence of sharp gradients.
2. Select initial circuit size:
 - 2–4 qubits for smooth PDEs,
 - 4–8 qubits for strongly nonlinear PDEs.
3. Start with a shallow circuit:
$$L = 1.$$
4. Increase qubits before increasing depth.
5. Evaluate:
 - PDE residual,
 - relative L_2 error,
 - gradient stability,
 - computational cost.
6. Increase depth only if additional expressivity is required.

This methodology emphasizes that the quantum layer should be designed according to the mathematical structure of the PDE rather than added as a generic neural network replacement.